# Map Reduce (1)

**<u>Question 3 Le programme ComptageMot.java est disponible dans votre répertoire. Il permet de compter pour chaque mot, le nombre d'occurrence dans un fichier</u>**

**1. Lisez le code, testez le programme afin de compter le nombre d'occurrence de chaque mot dans les fichiers (à charger sur HDFS) : fables fontaine.txt, shakespeare.txt, et les 451 fichiers du répertoire lyrics.**

Résultat : pour fables fontaine.txt

```
etudiant@cloud-41:~/tp4$ hadoop jar ComptageMot.jar ComptageMot /fables_fontaine.txt /fables_fontaine_result2024-
04-28 14:07:08,081 INFO client.RMProxy: Connecting to ResourceManager at cloud-41.depinfo.etu/192.168.4.41:8040
2024-04-28 14:07:11,919 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Imp
lement the Tool interface and execute your application with ToolRunner to remedy this.
2024-04-28 14:07:12,083 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/s
taging/etudiant/.staging/job_1714295117923_0002
2024-04-28 14:07:13,812 INFO input.FileInputFormat: Total input files to process : 1
2024-04-28 14:07:14,308 INFO mapreduce.JobSubmitter: number of splits:1
2024-04-28 14:07:14,674 INFO Configuration.deprecation: yarn.resourcemanager.system-metrics-publisher.enabled is
deprecated. Instead, use yarn.system-metrics-publisher.enabled
2024-04-28 14:07:15,736 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1714295117923_0002
2024-04-28 14:07:15,741 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-04-28 14:07:17,159 INFO conf.Configuration: resource-types.xml not found
2024-04-28 14:07:17,166 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2024-04-28 14:07:17,653 INFO impl.YarnClientImpl: Submitted application application_1714295117923_0002
2024-04-28 14:07:18,028 INFO mapreduce.Job: The url to track the job: http://cloud-41:8088/proxy/application_1714
295117923_0002/
2024-04-28 14:07:18,034 INFO mapreduce.Job: Running job: job_1714295117923_0002
2024-04-28 14:07:48,979 INFO mapreduce.Job: Job job_1714295117923_0002 running in uber mode : false
2024-04-28 14:07:49,091 INFO mapreduce.Job:  map 0% reduce 0%
2024-04-28 14:08:09,542 INFO mapreduce.Job:  map 100% reduce 0%
2024-04-28 14:10:01,772 INFO mapreduce.Job: Task Id : attempt_1714295117923_0002_r_000000_0, Status : FAILED
2024-04-28 14:10:28,187 INFO mapreduce.Job:  map 100% reduce 100%
2024-04-28 14:10:29,214 INFO mapreduce.Job: Job job_1714295117923_0002 completed successfully
2024-04-28 14:10:29,727 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=118104
                FILE: Number of bytes written=647133
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=63597
                HDFS: Number of bytes written=36788
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Failed reduce tasks=1
                Launched map tasks=1
                Launched reduce tasks=2
                Data-local map tasks=1
```

```
etudiant@cloud-41:~/tp4$ hadoop fs -ls /fables_fontaine_result
Found 2 items
-rw-r--r--   2 etudiant supergroup          0 2024-04-28 14:10 /fables_fontaine_result/_SUCCESS
-rw-r--r--   2 etudiant supergroup      36788 2024-04-28 14:10 /fables_fontaine_result/part-r-00000
etudiant@cloud-41:~/tp4$ hadoop fs -cat /fables_fontaine_result/part-r-00000
```

```
so      1
société 2
société,        1
soeur   1
soeur;  1
soeurs  1
soif;   1
soin    1
soin.   1
sois    1
soit    4
soldats,        1
soleil, 1
solicit 2
solicitation    1
solide. 1
somme   1
sommeil,        1
sommes  2
sommes, 1
sommes: 1
sommes; 1
son     36
songe   2
songeaient      1
songeant        1
songer  2
sonner  1
sonnette:       1
sont    21
sort    2
sort.»  1
sorte,  1
sortes  2
sortirait;      1
sortit  1
sot     1
sottise!        1
sottises        1
souci:  1
soudoyer        1
souffrir        1
```

Résultat : pour shakespeare.txt

```
etudiant@cloud-41:~$ hadoop jar ComptageMot.jar ComptageMot /shakespeare.txt /shakespeare_result
```

```
etudiant@cloud-41:~$ hadoop fs -ls /shakespeare_result
Found 2 items
-rw-r--r--   2 etudiant supergroup          0 2024-04-28 12:53 /shakespeare_result/_SUCCESS
-rw-r--r--   2 etudiant supergroup     746831 2024-04-28 12:53 /shakespeare_result/part-r-00000
```

```
etudiant@cloud-41:~$ hadoop fs -cat /shakespeare_result/part-r-00000
```

```
wounding,       2
wounding.       1
woundings       1
woundless       1
wounds  49
wounds! 4
wounds' 1
wounds, 15
wounds,--God    1
wounds. 2
wounds: 2
wounds; 3
wounds? 3
woven   4
wow.    1
wrack   1
wrack!  2
wrack,  1
wrack.  1
wrack;  1
wrackfull       1
wragged 1
wrangle 5
wrangle,        1
wrangle:        1
wrangler        1
wranglers,      1
wrangling       6
wrangling;      1
wrap    5
wrapp'd 4
wrapped 1
wraps   1
wrapt   3
wrath   25
```

**2. Modifiez le code pour rendre le programme insensible à la casse (par exemple en transformant tout en minuscule) et en ignorant les caractères de ponctuation (par exemple, en les utilisant également comme séparateurs dans StringTokenizer).**

Ajout de la fonction toLowerCase() et des ponctuations :

```java
public class ComptageMot {
  static class ComptageMotMapper extends Mapper <LongWritable, Text, Text, IntWritable> {
    public void map (LongWritable clef, Text valeur, Context contexte)
      throws IOException, InterruptedException {
        // traiter valeur (la ligne du fichier lue) pour en extraire ce qu'il faudra envoyer aux reducer
        // la clef (par defaut l'offset de la ligne en cours sur le fichier est rarement utilise.
    String ligne = valeur.toString () ;
    ligne = ligne.toLowerCase();
    StringTokenizer tokenizer = new StringTokenizer (ligne, " -_.,:;?!/()#$][*") ;
    while (tokenizer.hasMoreTokens ()) {
            contexte.write (new Text (tokenizer.nextToken ()), new IntWritable (1)) ;
    }
  }
  }
```

Résultat pour fables fontaine :

```
sentiment       2
sept    2
sera    6
serais  1
serait  4
seras   1
serrant 1
sers    1
servent 2
servi   3
service 1
servir  1
servit  1
ses     17
set     8
seul    4
seule   1
seulement       5
several 1
shall   2
share   1
shared  1
sharing 1
should  2
si      30
signalés        1
signifiait      1
```

**3. En vous inspirant du code précédent, écrivez un programme ComptageInitiale.java pour compter pour chaque lettre de l'alphabet, le nombre de mot commençant par cette lettre dans les fichiers**

Modification du mapper par rapport au code précédent :

```
public class ComptageInitiale {
  static class ComptageInitialeMapper extends Mapper <LongWritable, Text, Text, IntWritable> {
    public void map (LongWritable clef, Text valeur, Context contexte)
      throws IOException, InterruptedException {
        // traiter valeur (la ligne du fichier lue) pour en extraire ce qu'il faudra envoyer aux reducer
        // la clef (par defaut l'offset de la ligne en cours sur le fichier est rarement utilise.
    String ligne = valeur.toString () ;
    ligne = ligne.toLowerCase();
    StringTokenizer tokenizer = new StringTokenizer (ligne, " -_.,:;?!/()#$][*") ;
    while (tokenizer.hasMoreTokens ()) {
            String mot = tokenizer.nextToken();
            if (mot.length() > 0 && Character.isLetter(mot.charAt(0))) {
                char premiereLettre = mot.charAt(0);
                contexte.write (new Text (String.valueOf(premiereLettre)), new IntWritable (1)) ;
            }
    }
  }
  }
```

Résultat pour fables fontaine :

ComptageInitiale.java | ComptageMot.java | part-r-00000

```
a	670
b	214
c	726
d	867
e	571
f	408
g	200
h	91
i	315
j	157
k	6
l	904
m	453
n	311
o	381
p	810
q	387
r	243
s	607
t	711
u	163
v	242
w	206
x	1
y	96
z	2
à	124
â	5
é	60
ê	9
ô	5
```

si on vérifie le résultat de comptageMot, on avait bien 9 mot commençant par ê, 5 mots commençant pat ô, etc.:

```
éviter	1
êtes	3
être	6
ô	3
ôter	1
ôtez	1
```

**4 . En vous inspirant du code, écrivez un programme ComptageLettre.java pour compter pour chaque lettre de l'alphabet, le nombre d'occurrence de cette lettre dans les fichiers.**

```java
public class ComptageLettre {
    static class ComptageLettreMapper extends Mapper <LongWritable, Text, Text, IntWritable> {
        public void map (LongWritable clef, Text valeur, Context contexte) throws IOException,InterruptedException {
        // traiter valeur (la ligne du fichier lue) pour en extraire ce qu'il faudra envoyer aux reducer
        // la clef (par defaut l'offset de la ligne en cours sur le fichier est rarement utilise.
            String ligne = valeur.toString () ;
            ligne = ligne.toLowerCase();
            StringTokenizer tokenizer = new StringTokenizer (ligne, " -_.,:;?!/()#$][*") ;
            while (tokenizer.hasMoreTokens ()) {
                String mot = tokenizer.nextToken();
                for (int i = 0; i < mot.length(); i++){
                    char lettre = mot.charAt(i);
                    if (Character.isLetter(lettre)) {
                        contexte.write(new Text(Character.toString(lettre)), new IntWritable (1));
                    }
                }
            }
        }
    }
```

Résultat pour fables fontaine :

```
etudiant@cloud-41:~/tp4$ hadoop fs -cat /fables3_fontaine_result/part-r-00000
a       3231
b       557
c       1528
d       1535
e       6464
f       769
g       719
h       782
i       3149
j       280
k       135
l       2256
m       1196
n       3179
o       3057
p       1326
q       502
r       3129
s       3194
t       3664
u       2556
v       580
w       268
x       181
y       400
```

**Question 4**

**1. Pour chaque question, testez le programme sur les fichiers personnes*.txt générés dans le TD précédent. 1. Ecrivez un programme ComptageNom qui pour chaque nom de famille (3eme champ) distinct affiche le nombre de personne ayant ce nom.**

```java
public class ComptageNom {
    static class ComptageNomMapper extends Mapper <LongWritable, Text, Text, IntWritable> {
        public void map (LongWritable clef, Text valeur, Context contexte) throws IOException,InterruptedException {
        // traiter valeur (la ligne du fichier lue) pour en extraire ce qu'il faudra envoyer aux reducer
        // la clef (par defaut l'offset de la ligne en cours sur le fichier est rarement utilise.
            String ligne = valeur.toString () ;
            ligne = ligne.toLowerCase();
            StringTokenizer tokenizer = new StringTokenizer (ligne, ":") ;
            tokenizer.nextToken();
            tokenizer.nextToken();
            String nomFamille = tokenizer.nextToken().trim();
            contexte.write(new Text(nomFamille), new IntWritable (1));
        }
    }
```

Résultat avec personne2.txt contenant 200 000 entrées (attention) :

```
lecoq   155
lecourt 161
lecuyer 140
ledoux  168
ledru   148
leduc   164
lefebvre        163
lefeuvre        154
lefevre 177
lefort  165
lefranc 168
lefrancois      144
legay   140
legendre        146
leger   163
legrand 171
legras  163
legros  156
lejeune 140
leleu   166
lelievre        163
lelong  161
leloup  154
lemaire 162
lemaitre        151
lemarchand      164
lemercier       172
```

**2. Ecrivez un programme IdParNom qui pour chaque nom de famille distinct affiche la liste des identifiants (la liste des compteurs) des personnes ayant ce nom.**

Résultat pour fables fontaine :

```
public class IdParNom {
    static class IdParNomMapper extends Mapper <LongWritable, Text, Text, IntWritable> {
        public void map (LongWritable clef, Text valeur, Context contexte) throws IOException,InterruptedException {
        // traiter valeur (la ligne du fichier lue) pour en extraire ce qu'il faudra envoyer aux reducer
        // la clef (par defaut l'offset de la ligne en cours sur le fichier est rarement utilise.
            String ligne = valeur.toString () ;
            ligne = ligne.toLowerCase();
            StringTokenizer tokenizer = new StringTokenizer (ligne, ":") ;
            int compteur = Integer.parseInt(tokenizer.nextToken().trim());
            tokenizer.nextToken();  //Ignorer les prenoms
            String nomFamille = tokenizer.nextToken().trim();
            contexte.write(new Text(nomFamille), new IntWritable(compteur));
        }
    }


    public static class IdParNomReducer extends Reducer <Text, IntWritable, Text, Text> {
        public void reduce (Text clef, Iterable <IntWritable> valeurs, Context contexte) throws IOException,
InterruptedException {
            StringBuffer ids = new StringBuffer() ;
            for (IntWritable value : valeurs) {
                ids.append(value.toString()).append(" ");
            }
            contexte.write (clef, new Text(ids.toString())) ;
        }
    }
```

```
140201 61443 141108 44082 67116 26071 137483 85208 105610 175863 183276 54871 127875 6
3882 156949 175155 186357 83378 67540 83817 118111 160807 24893 162177 111271 72793 16
6039 105785 157830 133190 66010 18185 153286 167008 194713 145541
wolff   106582 86324 34166 34182 86483 115725 183790 194640 117633 116221 55550 118041
 194066 183885 129791 98179 7232 1881 100573 20131 198243 98565 43301 184619 19843 120
33 80805 56483 142668 56618 140703 131667 42398 95072 131833 167960 126355 189981 1303
27 74946 126073 46147 57300 81494 175816 78220 141628 166499 26439 166263 94055 73455
52212 165537 176465 57976 105603 72801 27386 27451 58330 185489 27799 139884 62698 176
750 72251 5489 99047 2871 164178 71899 124111 40699 83124 77933 136215 163184 133678 8
938 139259 91604 59149 123273 6804 10919 161301 186118 16813 39394 69461 195681 10802
69312 110529 110611 172321 44056 111138 143256 121938 96821 68627 150048 121798 159908
 96819 98448 13627 31542 111916 12794 193547 68076 20370 32096 150633 15956 15894 8960
9 22530 85405 32404 50387 137605 32612 120737 182425 85591 48360 66599 151333 238 6650
7 99922 120377 151541 86132 180523 180582 33688 199757 141336 180796
zimmermann      177196 54095 54050 54667 21679 142671 142687 21784 79531 21909 98920 4
4408 55067 128813 77838 20485 77734 44081 143287 141373 99914 197741 96519 170150 1007
18 20108 53373 131126 174334 55981 45718 127559 80608 184686 48981 101900 117903 19857
3 174962 102363 168569 175063 95293 64281 145331 46018 175268 167957 103011 167867 167
568 125975 81521 25675 81522 175796 125647 81584 185101 35175 183250 125520 73727 1967
93 104775 41685 52271 52241 93703 93317 136459 51813 164228 28478 196264 11222 147973
3609 40523 71152 51361 40279 11082 51265 177900 29466 7169 161507 122802 10806 69443 1
22482 149671 110618 16581 182662 63967 110987 60253 111084 3505 160372 31226 68502 508
3 121733 150303 159640 112029 4617 121422 67928 16009 179503 67534 112830 32199 150726
 134841 150741 199597 67137 89449 113615 151073 151290 66637 114145 120425 85939 61623
 135276 65995 156946 37601 86277 180889 34264 155906 137178 9740 155565 119374 119149
118310 198152 129871 129983 130093 48840
etudiant@cloud-41:~/tp4$
```